/spawn

Launch a named agent as a detached background sub-agent: /spawn <prompt…> | /spawn help

Overview

`/spawn` starts a background sub-agent directly from the chat input — no need to first ask the primary agent to call `new_agent`. The task is **detached**: it runs concurrently, appears in the Agents panel, and logs its completion, but nothing ever waits on it. It is not shown by `list_agents`, cannot be awaited via `wait_agents` (with or without `task_ids`), is untouched by `team_wait` (it lives on the sub-agent registry, not the team runtime), and its result is never injected back into the chat.

Use `/spawn` when all you want from a sub-agent are its **side effects** — writing a file, refreshing the index, running a diagnostic — and you do not want its reply body back in the conversation.

Syntax

```
/spawn <agent> <prompt...>
/spawn help           # same help text; also shown for a bare /spawn
```

- `<agent>` — any agent name `/agent` accepts: built-ins (`general`, `explore`, `build`, `plan`, `code-review`, …) and custom agents loaded from `.ragent/agents/` or the user-level directory. Resolution is case-insensitive; hidden built-ins resolve only by exact, case-sensitive name.
- `<prompt...>` — the full task text. It is passed to the sub-agent as its user turn; the sub-agent has no access to the parent's history, so be specific.

Behaviour

Property	Detached (`/spawn`)	Standard background (`new_agent` with `background: true`)
Runs concurrently	yes	yes
Agents panel entry during run	yes	yes
Appears in `list_agents`	**no**	yes
Awaitable via `wait_agents`	**no**	yes
Result injected into chat	**no**	yes
Report persisted to `log/subagents/<id>.md`	yes (on completion)	yes (on completion)
`SubagentComplete.finish_reason`	real loop outcome (`stop` / `truncation` / `length` / `cancelled` / `error`)	same
Cancellable via `/cancel <prefix>`	yes	yes

Because nothing ever reads the detached task's reply body, a `/spawn` prompt whose deliverable is a report MUST name a file for the sub-agent to write (e.g. `/spawn general … write ANTIPAT.md with …`)

— the Subagent completion protocol now requires the requested file to be written and verified before `agent_complete`, and the full final output is additionally persisted to `log/subagents/<task-id>.md` when the task finishes.

The sub-agent inherits the session's current provider/model. The launch is recorded in the `/spawn` slot; a second `/spawn` issued while the previous one is still registering is refused with a `spawn: another /spawn is still launching` warning (retry a moment later).

While the primary agent is mid-turn, `/spawn` is enqueued like any other slash command (input-queue counter increments) and runs at the next turn boundary.

Examples

```
/spawn explore Find every call site of `resolve_agent` and report the file list
```

Launches `explore` in the detached background. When it finishes the log panel records the completion; the chat does not receive the report body. The full report is read at `log/subagents/explore-<id>.md` with the `read` tool (or open it in another session via `/resume`).

```
/spawn general Write a one-paragraph summary of this repository to docs/one-line.md
```

The detached agent writes the file and exits; nothing further happens in this chat.

Related

- `/agent [<name>]` — change which agent drives the main session.
- `/agents` — list built-in and custom agents.
- `/cancel <id>` — cancel a running background task (including a detached one).
- `new_agent` tool — the LLM-facing equivalent; gained the optional `detached: true` parameter with the same semantics as `/spawn`.